Task 3

Train Multiple Machine Learning Models (scikit-learn)

Predictive Analytics & AI Model Deployment - Customer Churn Prediction

Objective. Train several ML models with scikit-learn and tune the most promising one.

1. Approach

- Six candidate models spanning linear, tree, ensemble, boosting and distance-based families, all with scikit-learn.
- Each model is wrapped in a **single Pipeline** (feature engineering -> preprocessing -> estimator) so cross-validation re-fits the preprocessing on every fold: no data leakage.
- **Stratified 3-fold cross-validation** on the training set, scored by ROC-AUC (threshold-independent and robust to the 27% class imbalance).
- Every fitted pipeline is persisted with joblib to `models/`.

2. Cross-validation results (training set)

Model	Key hyper-parameters	CV ROC-AUC (mean)	+/- std	Time (s)
Logistic Regression	max_iter=2000, C=1.0	0.7934	0.0049	0.6
Decision Tree	max_depth=7, min_samples_leaf=40	0.7533	0.0012	0.9
Random Forest	200 trees, min_samples_leaf=5, max_features=sqrt	0.7818	0.0025	15.6
Gradient Boosting (HistGB)	lr=0.06, 250 iterations	0.7868	0.0033	2.5
k-Nearest Neighbours	k=35, distance-weighted	0.7566	0.0031	1.9
Gradient Boosting - tuned	RandomizedSearchCV, 10 candidates x 3 folds	0.7907		19.2

Best tuned parameters for gradient boosting: l2_regularization=0.01648, learning_rate=0.06696, max_iter=150, max_leaf_nodes=8, min_samples_leaf=100.

3. Observations

- Logistic regression is the strongest single model in CV (0.793) - the churn signal in this dataset is largely additive, which a linear model captures well. Regularised, tuned boosting is close behind (0.791).
- The un-tuned decision tree and k-NN are weakest (0.753, 0.757): a single tree overfits and k-NN suffers in 44 encoded dimensions.
- Tuning helped boosting (0.787 -> 0.791) mainly by using shallower trees (8 leaves) and larger leaf sizes, i.e. more regularisation.
- Standard deviations across folds are small (about 0.001-0.005), so the ranking differences of >0.01 are meaningful.

4. Output

- `models/*.joblib` - trained pipelines.
- `results/task3_training.json`.

Appendix - Source code

src/train_models.py

```
"""TASK 3 - Train multiple ML models with scikit-learn.

Every model is a full pipeline (feature engineering + preprocessing + estimator), evaluated
with stratified 3-fold cross-validation on the training set, then fitted and saved.
A HistGradientBoosting model is additionally tuned with RandomizedSearchCV.
"""
import json
import time
import joblib
import pandas as pd
from scipy.stats import loguniform
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold, cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from .config import DATA_PROC, MODELS, RESULTS, SEED, TARGET
from .features import make_pipeline

CV = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)


def candidate_models():
    return {
        "logistic_regression": LogisticRegression(max_iter=2000, C=1.0),
        "decision_tree": DecisionTreeClassifier(max_depth=7, min_samples_leaf=40, random_state=SEED),
        "random_forest": RandomForestClassifier(n_estimators=200, min_samples_leaf=5, max_features="sqrt",
                                                n_jobs=-1, random_state=SEED),
        "gradient_boosting": HistGradientBoostingClassifier(learning_rate=0.06, max_iter=250,
                                                            random_state=SEED),
        "knn": KNeighborsClassifier(n_neighbors=35, weights="distance", n_jobs=-1),
    }


def load_train():
    train = pd.read_csv(DATA_PROC / "train.csv")
    return train.drop(columns=[TARGET]), train[TARGET]


def main():
    X, y = load_train()
    results = {}
    for name, est in candidate_models().items():
        pipe = make_pipeline(est)
        t0 = time.time()
        scores = cross_val_score(pipe, X, y, cv=CV, scoring="roc_auc")
        pipe.fit(X, y)
        results[name] = {"cv_roc_auc_mean": round(float(scores.mean()), 4),
                         "cv_roc_auc_std": round(float(scores.std()), 4),
                         "train_seconds": round(time.time() - t0, 1)}
        joblib.dump(pipe, MODELS / f"{name}.joblib")
        print(name, results[name], flush=True)

    # hyper-parameter tuning
    t0 = time.time()
    search = RandomizedSearchCV(
        make_pipeline(HistGradientBoostingClassifier(random_state=SEED)),
        {"model__learning_rate": loguniform(0.02, 0.2), "model__max_iter": [150, 250, 400],
         "model__max_leaf_nodes": [8, 15, 31], "model__min_samples_leaf": [20, 50, 100],
         "model__l2_regularization": loguniform(1e-3, 10)},
        n_iter=10, cv=CV, scoring="roc_auc", random_state=SEED, n_jobs=1, refit=True)
    search.fit(X, y)
    joblib.dump(search.best_estimator_, MODELS / "gradient_boosting_tuned.joblib")
    results["gradient_boosting_tuned"] = {
        "cv_roc_auc_mean": round(float(search.best_score_), 4), "cv_roc_auc_std": None,
        "train_seconds": round(time.time() - t0, 1),
        "best_params": {k.replace("model__", ""): (round(v, 5) if isinstance(v, float) else v)
                        for k, v in search.best_params_.items()}}
    print("tuned", results["gradient_boosting_tuned"], flush=True)
    (RESULTS / "task3_training.json").write_text(json.dumps(results, indent=2))
if __name__ == "__main__":
    main()
```